

Single Player Freckers Report

Search Strategy and Implementation Details

This implementation uses A-star Search to find the path for a single red frog to move from the initial row (near row 0) to the bottom row (row 7). Using Python's 'heap q' as the priority queue, where a triplet of (f_value, unique_id, position) is placed, and the smaller the f_value, the more likely it is to be retrieved. We also used close_list which is set to record the node we have visited to prevent visiting existing node. And a dictionary called g_values to record the actual cost from the starting position to current position, and another dictionary called came_from is used to track the predecessor nodes and corresponding movements required for path reconstruction to return as our answer.

Assuming in the worst-case scenario, it may be necessary to traverse the vast majority of feasible chessboard states. If used 'b' represents the branching factor (the number of next steps that can be unfolded at each position), 'd' represents the search depth, and the theoretical time complexity of A-star is approximately $O(b^d)$. The spatial complexity is usually on the same level, as it requires storing all expanded nodes.

The worst-case time complexity looks the same as Breadth First Search or Iterative Deepening Search, however due to the use of heuristics, A-star can be more efficient in many cases, and can actually significantly reduce the number of nodes that need to be expanded.

Design of Heuristic Function

The goal of heuristic function is to predict the cost is large enough but cannot exceed the actual cost which is admissible. We compute the maximum row that can be reached in a single move from the current state, then estimate how many moves are needed to reach row 7.

In detail, we use get_possible_moves function to obtain all possible positions that can be reached in the next step from the current position, find the position with the largest row coordinate. Finally, take $(7 - \text{current_row})$, divide it by this maximum possible advancement, and apply the ceiling function to determine the estimated number of moves required.

Six Red Frogs Scenario

The time and space complexity grows exponentially. If only one frog can be moved at each step, the branching factor may be approximately " $6 \text{ (number of selectable frogs)} \times b \text{ (number of ways a single frog can move)}$ ", that is $6b$. In the worst-case scenario, the search tree depth may approach the total number of steps taken by each of the six frogs.

In the case of multiple red frogs, nodes are no longer just the coordinates of individual frogs, but the state of the entire chessboard, this means what is stored in heapq is no longer $(f_value, unique_id, position)$, but $(f_value, unique_id, board_state)$. This state will include the positions of all frogs (whether red or blue) on the chessboard, thus fully reflecting all possible movements and their interactions.

And we will choose **Multi-Agent Path Finding** to consider the collaboration conditions.

Conflict Based Search (CBS): First, independently run A* search for each red frog to obtain preliminary paths, and then detect conflicts or collaborations in these paths. For these parts, additional constraints are introduced and re planned to gradually combine the collaborations or eliminate conflicts, and obtain the overall optimal solution.

Since the goal changes from one red frog reaches row 7 to all six red frogs reach row 7. We can still use A-star search, but we need to redesign our heuristic function by considering the 'collective' cost of multiple red frogs simultaneously.

We can still compute each red frog's individual optimal cost, **h_{single}**, which is the minimum number of moves required for that frog to reach the target row (row 7) from its current position. Thus, we define the base cost as $H_{base} = \sum_{i=1}^6 h_{single}(i)$. Next, we define a cooperative benefit function **C_{coop}(n)** that estimates the minimum number of moves that can be saved through cooperation in the current global state. **C_{coop}(n)** can be estimated using the following approaches:

1. **Path Overlap :**

For each red frog, precompute an individual optimal path and examine the overlapping segments among these paths. The length of these overlapping segments can serve as an upper bound on the number of "shared moves" that can potentially be saved.

2. **Cooperative Potential :**

Analyze the current state to identify red frogs with significant cooperative potential. For instance, the movement of one frog might clear a path or open up a more direct route for another frog, effectively translating into saved moves.

The value of **C_{coop}(n)** should then be set as the lower bound of the number of moves saved, as determined by the estimates from these two methods.

The heuristic function obtained by combining the basic estimation with the cooperative benefits is: $h(n) = H_{base} - C_{coop}(n)$